



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E
WEB PROGRAMMING
AND INTERFACE DESIGN

OVERVIEW

- What is Javascript?
- Java vs Javascript
- Javascript syntax
- Examples
- Questions and Discussion

REVISION QUESTION

What will happen in this case?

```
<p style="color green">text</p>
```

- a) make that specific paragraph green
- b) error
- c) blank screen
- d) none of the above

ANSWER

Answer: **d)**

- Text will be displayed without style
- formatting (i.e., 'Text') – as the 'colon :' is missing

REVISION QUESTION

What is the Form attribute type for hiding data:

- a) secret
- b) blank
- c) hidden
- d) passive
- e) mask

ANSWER

Answer: **c) hidden**

```
<input type="hidden" name="pagename" value="10" />
```

WHY IS JAVASCRIPT CALLED JAVASCRIPT?



JAVASCRIPT VS JAVA

- Javascript is different from Java
 - **similar syntax**
 - both descend from C
 - Java is **strongly typed**
 - Java is **compiled** (into byte code)
 - actually hybrid because byte code is mostly interpreted
 - Java has strong **Object Orientated** foundation
 - in Javascript, OO was an afterthought
- Name was primarily taken as a **marketing** decision
- **You should already know Java**

WHERE DID JAVASCRIPT COME FROM?

- JavaScript is a self inspired scripting language that
- *Brendan Eich** developed for **Netscape** in 1995
- Supports object-oriented and imperative programming styles was **standardised as ECMAScript** 1 in 1997, 2 in 1998, 3 in 1999

JAVASCRIPT SYNTAX

JavaScript data types

- **string** sequence of 16 bit UTF-16 characters
- **number** type of double precision 64 bit IEEE for mat number
- **boolean** type having value of true or false
- number
- **function** declaration

-- These 2 types cannot contain values

- **undefined** sole value possessed by unassigned variables
- **null** value denoting intentional lack of a value

-- Everything else is an object

JAVASCRIPT SYNTAX

The **typeof** operator is used to get the data type (**returns a string**) of its operand

```
typeof(4+7);    //returns number  
typeof("47");  //returns string
```

Six possible values that **typeof** returns: object, boolean, function, number, string, and undefined

REVISION QUESTION

Which primitive type represents integers in Javascript?

- a) int
- b) integer
- c) number
- d) double
- e) var

ANSWER

Answer: **c) number**

JAVASCRIPT SYNTAX

Objects are members of the type Object that have

- prototype object
- extensible enumerable list of properties (including functions)
- include keyed, indexed collection, function types

Variables are **case-sensitive** identifiers that

- start with alphabetic letter, underscore or dollar sign
- contain only alphanumerics, dollar signs and underscore
- have no type - type belongs to their value

JAVASCRIPT SYNTAX

- Keyword **var** declares variables with scope of function they're in.
- In ES 6+ let and const declare **block-scoped** variables.
- **let** variables can't be redeclared in scope. var variables can.
- **const** variables can index loops but can't be reassigned values.

```
var i, number_of_hits = 0, done = true, fruit = "yoghurt";  
let ID = Symbol();  
const pi = Math.PI;
```

- Undeclared variable has global scope unless declared after use.

QUESTION

Javascript variable identifiers are case-sensitive?

- a) true
- b) false

ANSWER

Answer: **a) true**

```
var Cat;  
var cat;
```

JAVASCRIPT STATEMENTS

- JavaScript statement ends in **";"**. Newline forces their interpolation.
-
- JavaScript statements are composed of
 - **comments** single start with `"//"`, multiline inside `"/*"` and `"*/"`
 - expressions compounds of values, variables and operators
 - **keywords** reserved words like `"function"`, `"var"`, `"new"` etc.
 - operators (see next slide)
 - values primitive values and objects
 - variables identifiers

EXPRESSIONS

- Expressions can contain various types of operators
- Operators:

```
- arithmetic +, -, *, /, % 3+4-6/7*8
- assignment =, =+, =*, =/, -= E=M*C*C
- bitwise &, ^, |, <<, >> 15 ^ 9
- comparison >, >=, <, ==, !=, === x>y+z
- logical &&, ||, ! age >= 0 && age < 120
- string + "Let there" + " be light!"
- conditional cond ? val1 : val2 age > 17 ? "adult" : "mi
```

QUESTION

What does the following Javascript output?

```
var a = 22;  
a *= 3;  
alert( a );
```

ANSWER

Answer: 66

SPECIAL IDENTIFIERS

- JavaScript has a few special identifiers including
 - **NaN** Not a Number - special IEEE for mat value
 - **Infinity** positive infinity number value
- JavaScript has **built-in objects** incl. Array, Date, Math, RegExp, String
- Engines may have built-ins for **GUI, IO, OS** - Navigator, Window

JAVASCRIPT CONTROL STRUCTURES

- JavaScript supports various programming language control structures:
 - **conditionals** if, if else, switch
 - **loops** for, for of, for in, do while, while
 - **functions** with arguments that return a value

EXAMPLES OF **CONDITIONAL** EXPRESSIONS

```
if ( s.length > 10 ) {  
    s=s.substring(0, 10);  
} else {  
    s=s.trim();  
}  
  
switch ( Math.floor(Math.random()*3) ) {  
    case 1: col = "red";  
    break;  
    case 2: col = "blue";  
    break;  
    default: col = "green";  
}
```

EXAMPLES OF **LOOPS**

- for loop

```
let sum = 0;
let arr = [1, 2, 3, 5, 7, 11, 13];
for (let i=0; i<arr.length; i++) { // classic loop
    sum += arr[i];
}

for (let item of arr) { sum += item; } // loop over collection

var car = { make: "Ford", model: "Focus", year: 2017 };
for (var p in car) { // enumerable properties loop
    if ( isNaN(car[p]) ) car[p] = car[p].toUpperCase();
}
```

EXAMPLES OF **LOOPS**

- do while
- while

```
do {  
    var j = page.indexOf( "=", j );  
    if ( j != -1 )  
        s += page.substring( j, j + 2 );  
} while ( j != -1 && j < len );  
  
var count = 0;  
while ( ( i = page.indexOf( "href=", i ) ) != -1 ) {  
    count++;  
}
```

QUESTION

What does the following Javascript code output?

```
let sum = 0;
let arr = [1, 2, 3];
for (let i=1; i<arr.length; i++) { // classic loop
    sum += arr[i];
}
alert( sum );
```

ANSWER

Answer: 5

```
Loop starts at `1` and sums the array values
```

STANDALONE JAVASCRIPT

- JavaScript can be run by standalone engines or within browsers.
- **Nashorn** is standalone JavaScript engine (ES5) released in Java 1.8
 - based on the Da Vinci Machine
 - implementing JSR-223 using new code base
- Nashorn engine is launched with **"jjs"** or "jrunscript" command.

```
jjs> abc = "cat";  
jjs> print( abc + " hellow" );  
jjs> quit()
```

JAVASCRIPT FUNCTIONS

- JavaScript functions are first class objects where
 - **function keyword** has **optional name**
 - **arguments** are enclosed in (...)
 - arguments are passed by value
 - arguments may pass objects (e.g. functions) by reference
 - body in { ... }

```
function fn ( optionalParams ) { ... }
```

JAVASCRIPT FUNCTIONS

- Suppose isFruit() is declared in **fruit.js**:

```
function isFruit(x) {  
    if ( x == "apple" || x == "fig" || x == "pear" )  
    { return true; }  
    return false;  
}
```

- Function can be used by loading its definition and invoking it:

```
jjs> load("fruit.js")
```

JAVASCRIPT FUNCTIONS

- **Anonymous** functions are defined inline and may be assigned to a name
 - passed as argument in a function call
 - **self-invoked** i.e. applied inline to arguments
- Examples would be

```
anon = function() { print('I am anonymous'); }  
anon();
```

```
invoke = function(fn, arg) { print(fn(arg)); }  
invoke(function(i) {return i+1;}, 5 );
```

JAVASCRIPT FUNCTIONS

```
(function(s) { print('Say ', s); })("Hello")
```

- Builtin functions
 - Example, builtin objects like Math support standard functions like

```
Math.random( )
```

JAVASCRIPT STRING HANDLING

Why is string handling important?

JAVASCRIPT STRING HANDLING

Methods and properties for handling strings include

- **charAt(N)** returns character at index N
- **indexOf(S)** returns index of S in string
- **lastIndexOf(S)** returns last index of S in string
- **length** property storing length of string
- **split(S)** returns array of substrings separated by S
- **substring(M)** returns substring from index to end
- **substring(M, N)** returns substring between given indices
- **toLowerCase()** maps all letters to lower case
- **toUpperCase()** maps all letters to upper case
- **trim()** removes whitespace from both ends of string

QUESTION

How would you concatenate two Javascript strings?

```
var a = "cat";  
var b = "bat";  
// how to combine them into a new string "catbat"?
```

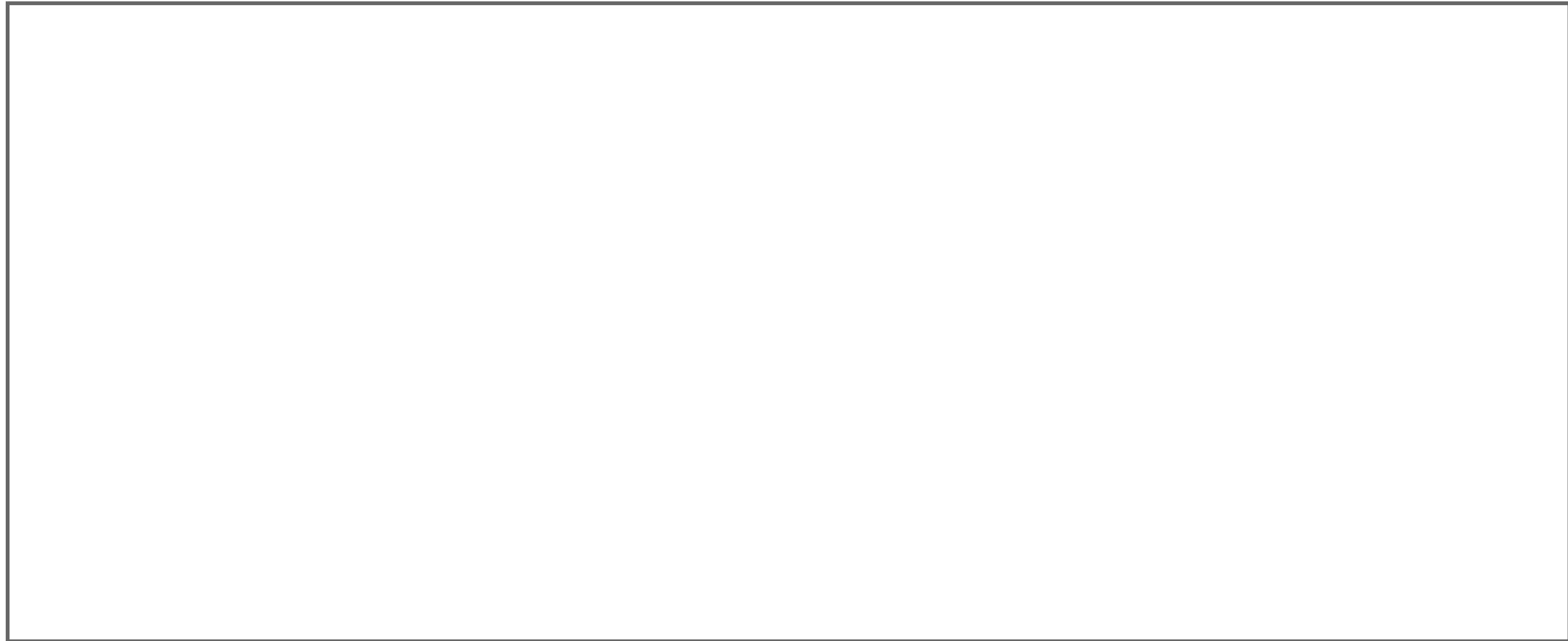
ANSWER

Answer: **a + b**

Note there is also:

```
str.concat(string2, string3, string4,....., stringN)
```

EXAMPLE FUNCTION IN URL.JS SHOWS HOW TO **NORMALISE A URL:**



JAVASCRIPT ARRAYS

- JavaScript Array is created using Array built-in

```
var cars = new Array();  
cars[0] = "Honda Civic";  
cars[1] = "Ford Focus";  
cars[2] = "Fiat Punto";
```

- Same array can also be created by either of

```
cars = new Array("Honda Civic", "Ford Focus", "Fiat Punto");  
cars = ["Honda Civic", "Ford Focus", "Fiat Punto"];
```

JAVASCRIPT ARRAYS

- Useful properties for an Array include:
 - **length** number of elements
 - prototype property allowing Array object to be extended

```
var cars = [ "Honda Civic", "Ford Focus", "Fiat Punto" ]  
cars.length  
> 3
```

```
Array.prototype.make = function(i) {return this[i].split(" ")[0]}  
cars.make(0)  
> Honda
```

QUESTION

What does the following Javascript code output?

```
let arr = ['a', 2, 'c'];  
alert( arr[9] );
```

ANSWER

Answer: **undefined**

```
array is out of bounds 0 to 2
```

JAVASCRIPT ARRAYS

Useful functions for an Array include

- **indexOf**(element) index of element
- **push**(element) add element to end and return new length
- **splice**(i, n, a, ..., k) replace n elements from index i with a ... k

JAVASCRIPT ARRAYS EXAMPLE

```
var cars = [ "Honda Civic", "Ford Focus", "Fiat Punto" ]
cars.indexOf("Fiat Punto")
> 2

["a", "b", "c", "a"].lastIndexOf("a")
> 3

cars.push("Fiat Tipo")
> 4

cars.splice(1, 2, "Rolls Royce")
> Ford Focus,Fiat Punto

cars
> Honda Civic,Rolls Royce,Fiat Tipo
```

JAVASCRIPT ARRAYS

- In 2011 3 builtin functions were added to **simplify array** handling.
- Each applies a function to array elements to implement their effect.
 - **filter()** selects out array elements meeting a condition
 - **map()** applies function to each array element
 - **reduce()** applies accumulator/concatenator across array elements

JAVASCRIPT ARRAYS

- **Duplicate** elements in array can be removed by filter() thus:

```
var bag = [4, 1, 4, 7, 1, 2]
bag.filter(function(e, i, ar) { return i === ar.indexOf(e); })

> 4,1,7,2
```

- **map()** has same general syntax as filter() operation.

JAVASCRIPT OBJECTS

- JavaScript objects are property **collections** that
 - are **non-primitive** values
 - have prototypes that are also objects
 - inherit their properties and methods from their prototype
- **All** objects **inherit from prototype** called Object.prototype

JAVASCRIPT OBJECT **EXAMPLES**

- Create an object with new keyword and assign properties to it:

```
vehicle = new Object();  
vehicle.make = "Ford";  
vehicle.model = "Focus";  
vehicle.year = 2012;
```

- Vehicle can also be declared and initialised **directly**

```
vehicle = { make: "Ford", model: "Focus", year: 2012 };
```

JAVASCRIPT OBJECT EXAMPLES

- **Associative array syntax** in JavaScript can also create vehicle

```
vehicle["make"] = "Ford";  
vehicle["model"] = "Focus";  
vehicle["year"] = 2012;
```

- Object prototype is standardly created by a constructor function.
 - **car.js** defines a constructor for object instances

```
function car(make, model, year) {  
  this.make = make;  
  this.model = model;  
  this.year = year;  
}
```

- that can be used with new keyword in expressions such as

```
vehicle = new car("lamborghini", "gallardo", 2013);
```

JAVASCRIPT OBJECT

- Object's properties can be listed by (e.g., getProps.js)

```
function getProps(obj) {  
  var props = [];  
  for (var p in obj) { props.push(p); }  
  return props;  
}
```

- Car prototype can be **dynamically** changed to set default values

JAVASCRIPT OBJECT

- **Functions** can be **added** to object prototype as follows

```
car.prototype.name = function() { return this.make + " " +  
this.model;}  
wheels.name() // invoke  
> Jaguar E-type
```

- Keyword `this` refers to object that owns JavaScript it occurs in
 - in function `this` refers to object that owns function
 - in object `this` refers to object itself
- Global object has some useful meta-programming functions
 - `eval()` evaluates string argument and executes it as code
 - `isNaN()` determines whether argument is not a number

JAVASCRIPT OBJECT (CLASS)

ES 6 introduced **class notation** to JavaScript where

- classes are syntactic sugar over **prototype-based objects**
- no new object-oriented model is introduced

Example class declaration **circle.js** can define a class function

```
class Circle {  
  constructor(radius, x, y) { this.radius = radius;  
    this.x = x; this.y = y; }  
  area() { return Math.PI * Math.pow(this.radius, 2); }  
  static info() { console.log("I know about circles"); }  
}
```

- A class expression (with/without a name) can also define it

```
var ring = class Circle { ... }
```

JAVASCRIPT OBJECT **EXAMPLE**

- First version of class

```
Circle.info()  
> I know about circles  
let ring = new Circle(10, 0, 0)  
ring.area()  
> 314.1592653589793
```

- **nicecircle.js** shows how ES 6+ classes can be **subclassed**

```
class NiceCircle extends Circle {  
  constructor(radius, colour) {  
    super(radius, 0, 0);  
    this.colour = colour;  
  }  
}
```

JAVASCRIPT OBJECT EXAMPLE

- **Extended** version of class
 - nicecircle.js shows how ES 6+ classes can be subclassed

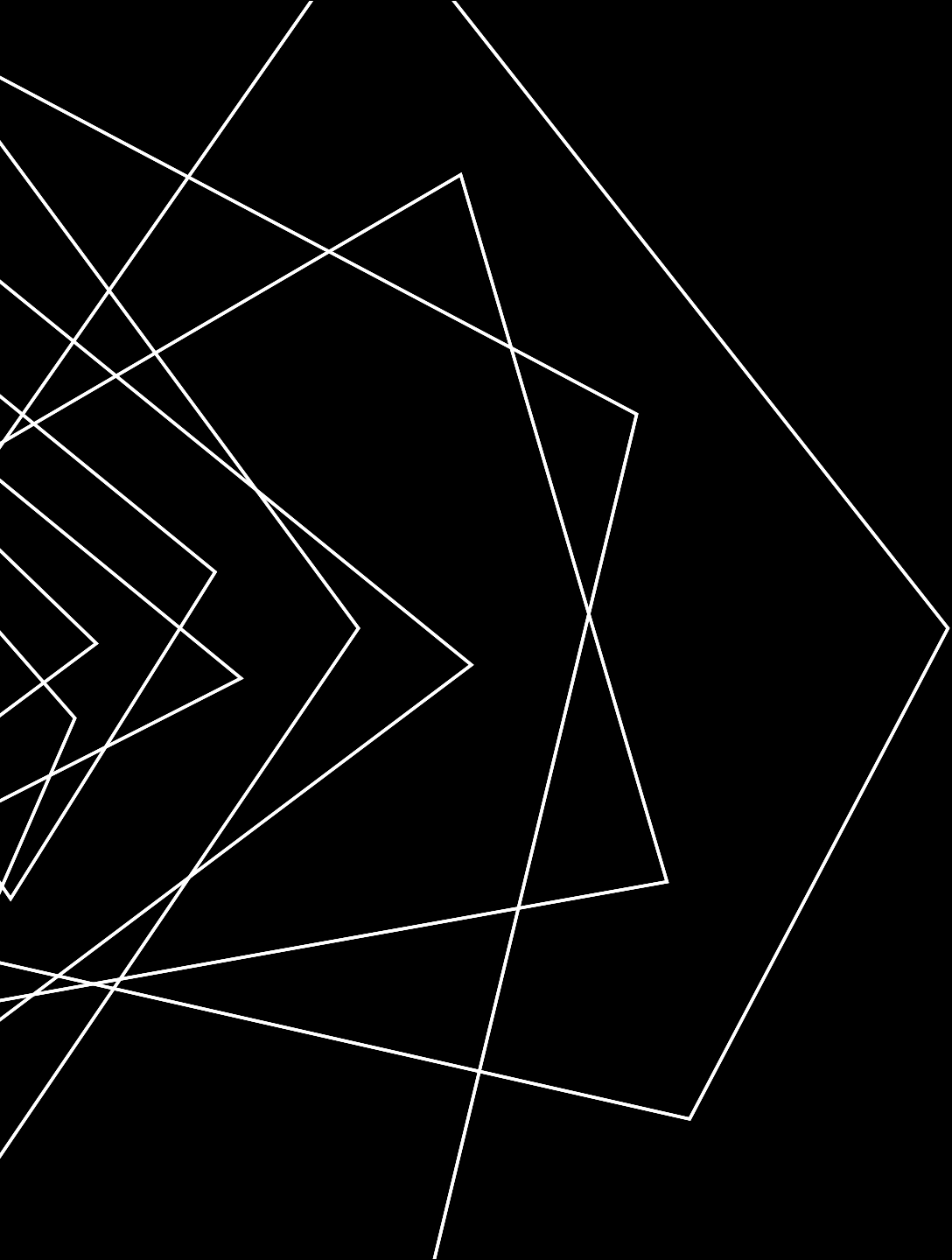
```
class NiceCircle extends Circle {  
  constructor(radius, colour) {  
    super(radius, 0, 0);  
    this.colour = colour;  
  }  
}
```

- In derived classes **super()** must be called before this.

```
var hoop = new NiceCircle(12, "red")  
hoop.area()  
> 452.3893421169302
```

SUMMARY

- Understand Javascript (**Basics**)
- JavaScript is a **weakly typed**, object-based (prototypes) language, standardised as ECMAScript
- JavaScript has a C-like syntax, is multi-threaded, **supports arrays, functions, objects** and typical procedural control structures.
- Primitive types are booleans, numbers, strings, symbols, null and undefined. Everything else is an object.



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335